

UNIVERZITET U SARAJEVU
ELEKTROTEHNIČKI FAKULTET
ODSJEK ZA RAČUNARSTVO I INFORMATIKU

Heuristički metod simuliranog kaljenja

SEMINARSKI RAD

- PRVI CIKLUS STUDIJA -

Studenti:
Ahmed Spahić, Sarah Mrakić

Mentor:
Red. prof. dr Željko Jurić, dipl. ing. el.

Sarajevo,
Februar 2026

Sažetak

Problem traženja minimuma i maksimuma proizvoljne funkcije predstavlja jedan od temeljnih problema numeričkih algoritama. Za ovaj problem još ne postoji, a vjerovatno neće ni postojati, univerzalan egzaktan algoritam koji je u općem slučaju brži od traženja grubom silom. Glavnu prepreku pri rješavanju predstavljaju lokalni minimumi, jer algoritam nikada sa sigurnošću ne može znati da li se nalazi u globalnom minimumu, osim ako ne istraži sve lokalne minimume. Potrebno je razviti način za pomjeranje iz lokalnog minimuma koji će u većini slučajeva odlučiti ispravno - upravo u tome se krije suštinska težina ovog problema.

Iako efikasan egzaktan algoritam ne postoji, razvijeni su mnogi heuristički i metaheuristički algoritmi za tu svrhu. Heuristički algoritmi ne daju uvijek optimalno rješenje, ali pronalaze zadovoljavajuća rješenja u razumnom vremenu, oslanjajući se pritom na specifičnosti konkretnog problema. Metaheuristički algoritmi dijele iste osobine, ali nisu vezani za specifičnost problema te se mogu primjenjivati na znatno širi skup problema.

Simulirano kaljenje, koje se u zavisnosti od literature svrstava u jednu ili drugu kategoriju, jedan je od najpoznatijih algoritama ovog tipa. Primjenjivo je na širok spektar optimizacijskih problema, a njegova implementacija nije toliko zahtjevna. Ovaj rad za cilj ima predstaviti osnovni koncept simuliranog kaljenja, detaljno razraditi korake algoritma te objasniti i demonstrirati neke od njegovih primjena. Nakon teoretskog uvoda slijedi implementacija algoritma u programskom jeziku Julia.

Abstract

The problem of finding the minimum or maximum of an arbitrary function is one of the fundamental problems in numerical algorithms. For this problem, there is still no universal exact algorithm - and most likely there never will be one - that is, in the general case, faster than brute-force search. The main obstacle in solving it is the presence of local minima, since an algorithm can never be certain that it has reached the global minimum unless it explores all local minima. Therefore, it is necessary to develop a mechanism for escaping local minima that will, in most cases, make the correct decision - this is precisely where the intrinsic difficulty of the problem lies.

Although an efficient exact algorithm does not exist, many heuristic and metaheuristic algorithms have been developed for this purpose. Heuristic algorithms do not always guarantee an optimal solution, but they find satisfactory solutions in reasonable time by exploiting the specific features of a given problem. Metaheuristic algorithms share these characteristics, but they are not tied to a particular problem structure and can therefore be applied to a much broader class of problems.

Simulated annealing, which depending on the literature is classified into one or the other category, is one of the most well-known algorithms of this type. It can be applied to a wide range of optimization problems, and its implementation is not overly complex. The aim of this paper is to present the basic concept of simulated annealing, to elaborate the steps of the algorithm in detail, and to explain and demonstrate some of its applications. Following the theoretical introduction, an implementation of the algorithm in the Julia programming language is presented.

Sadržaj

1 Teoretski uvod **1**
- 1.1 Motivacija i fizikalna analogija . . . 1
- 1.2 Metropolisov kriterij . . . 1
- 1.3 Raspored hlađenja . . . 2
- 1.4 Primjena na funkciju dvije promjenljive . . . 2

2 Implementacija algoritma **3**
- 2.1 Izvorni kod . . . 3
- 2.2 Struktura koda . . . 4
- 2.3 Testne funkcije . . . 4
- 2.4 Rezultati testiranja . . . 6
  - 2.4.1 Zaključci o tačnosti . . . 7

3 Primjene u praksi **8**
- 3.1 Problem trgovačkog putnika . . . 8
  - 3.1.1 Konfiguracija i prikaz rješenja . . . 8
  - 3.1.2 Operator susjedstva . . . 8
  - 3.1.3 Funkcija cilja . . . 9
  - 3.1.4 Raspored hlađenja . . . 9
  - 3.1.5 Testni primjer . . . 9
  - 3.1.6 Testiranje na Berlin52 instanci . . . 9
- 3.2 Problem ruksaka . . . 10
- 3.3 Optimizacija neuronskih mreža . . . 11

4 Zaključak i diskusija **12**

Literatura **13**

Poglavlje 1

Teoretski uvod

1.1 Motivacija i fizikalna analogija

Simulirano kaljenje (engl. *Simulated Annealing*, SA) jedna je od najpoznatijih (meta)heurističkih metoda za rješavanje globalnih optimizacijskih problema čija funkcija cilja nije eksplicitno zadana, već se vrednuje putem računarske simulacije [1]. Metodu su 1983. godine uveli Kirkpatrick, Gelatt i Vecchi, temeljući je na analogiji s fizikalnim procesom kaljenja materijala [2].

Kaljenje (engl. *annealing*) je toplinska obrada u kojoj se tvar zagrijava do taljenja, a zatim sporo hladi prema kontroliranom rasporedu. Pri visokim temperaturama čestice su raspoređene nasumično, dok sporim hlađenjem sustav dostiže stanje minimalne energije - kristalnu strukturu. Nagli pad temperature (*hardening*) rezultira metastabilnim stanjem s neminimalnom energijom. Analogija s optimizacijom uspostavlja se sljedećim preslikavanjima:

Fizikalni sistem	**Optimizacijski problem**
Stanje sistem	Kandidat rješenja
Energija stanja	Vrijednost funkcije cilja
Temperatura	Upravljački parametar c
Globalni energetski minimum	Globalni optimum

1.2 Metropolisov kriterij

Ključni gradivni element SA jest **Metropolisov algoritam** (Metropolis, Rosenbluth i Teller, 1953.), koji generiše sekvencu stanja sistem simulirajući fizikalni proces žarenja [1]. U kontekstu optimizacije, kriterij prihvaćanja novog rješenja j iz trenutnog rješenja i definisan je sa vjerovatnoćom:

$$P(\text{prihvati } j) = \begin{cases} 1, & \text{ako } f(j) < f(i) \\ e^{(f(i)-f(j))/c}, & \text{inače} \end{cases} \tag{1.1}$$

gdje je c upravljački parametar (temperatura), a $\Delta f = f(j) - f(i) \geq 0$ predstavlja pogoršanje funkcije cilja. Bolje rješenje uvijek se prihvata, dok se lošije prihvata s vjerovatnoćom koja eksponencijalno opada s porastom pogoršanja i padom temperature. Na taj način algoritam u ranim fazama slobodno istražuje prostor rješenja izbjegavajući lokalne optimume, a u kasnijim fazama konvergira prema kvalitetnom rješenju.

1.3 Raspored hlađenja

Strategija smanjivanja temperature c_k kroz iteracije naziva se **rasporedom hlađenja** (engl. *cooling schedule*). Najčešće korišten je *geometrijski raspored*:

$$c_{k+1} = \alpha \cdot c_k, \quad \alpha \in (0,1) \tag{1.2}$$

gdje je α faktor hlađenja, tipično iz intervala $[0.8, 0.99]$. Teorijski garanciju konvergencije prema globalnom optimumu pruža *logaritamski raspored* $c_k = c_0/\ln(1+k)$, ali je u praksi presložen [1]. Izbor rasporeda hlađenja, početne temperature c_0 te dužine Markovljevih lanaca L_k (broj prijelaza pri svakoj temperaturi) ključni su parametri koji izravno utječu na kvalitetu rješenja i brzinu konvergencije algoritma.

1.4 Primjena na funkciju dvije promjenljive

Neka je data funkcija dvije promjenljive $f(x_1,x_2)$ i neka se traži globalni minimum. Budući da je traženje maksimuma svodljivo na traženje minimuma negirane funkcije $-f(x_1,x_2)$, u nastavku se razmatra isključivo problem minimizacije. Kako je SA heuristički algoritam koji zahtijeva veliki broj iteracija, postupak se opisuje samo kroz korake u općem slučaju.

1. **Kreiranje energetske funkcije.** Za funkciju dvije promjenljive, energetska funkcija izravno odgovara funkciji cilja: $E(x_1,x_2) = f(x_1,x_2)$.

2. **Generisanje početnog rješenja $\mathbf{x}_0 = (x_1^{(0)}, x_2^{(0)})$.** Početno rješenje se obično odabire nasumično iz domene pretrage, uz poštivanje eventualnih ograničenja.

3. **Kreiranje susjednog rješenja.** Susjedno rješenje $\mathbf{x}' = (x_1', x_2')$ generira se perturbacijom trenutnog rješenja:
$$\mathbf{x}' = \mathbf{x} + \Delta\mathbf{x}, \quad \Delta\mathbf{x} \in (-\delta, \delta)^2 \tag{1.3}$$
gdje se svaka komponenta pomaka Δx_i odabire nezavisno i nasumično iz intervala $(-\delta, \delta)$. Novo rješenje se projektuje na zadani opseg pretrage kako bi ostalo dopustivo. Ako je $E(\mathbf{x}') < E(\mathbf{x})$, rješenje $\mathbf{x}'$ se usvaja. U suprotnom, $\mathbf{x}'$ se prihvata s vjerovatnoćom:
$$p = \exp\left[-\frac{E(\mathbf{x}') - E(\mathbf{x})}{T}\right] \tag{1.4}$$

4. **Provjera kriterija zaustavljanja.** Ukoliko je trenutno rješenje zadovoljavajuće, dostignut je maksimalni broj iteracija N, ili je promjena vrijednosti funkcije između dvije sukcesivne iteracije manja od ε, algoritam terminira.

5. **Hlađenje i ponavljanje.** Temperatura T se smanjuje prema odabranom rasporedu hlađenja, brojač iteracija se povećava, te se postupak nastavlja od trećeg koraka.

Nakon terminiranja, pod uvjetom ispravno odabranog energetskog modela i parametara hlađenja, dobiva se prihvatljivo rješenje u vremenu znatno kraćem od potpunog pretraživanja prostora .

Generalzacija na više promjenjivih se može uraditi na sličan način, samo je potrebno randomizirano birati više varijabli.

Poglavlje 2

Implementacija algoritma

2.1 Izvorni kod

Implementacija algoritma simuliranog kaljenja realizirana je u programskom jeziku Julia. Potpis funkcije odgovara specifikaciji datoj u laboratorijskoj vježbi 5, na predmetu "Optimizacija resursa" [3]:

```
SA(f, opseg, x0, delta_x, N, eps, T0; alpha=0.995)
```

gdje su parametri:

- `f` – funkcija cilja oblika $f(x_1, x_2)$,
- `opseg` – opseg pretraživanja oblika `[[xmin,xmax],[ymin,ymax]]`,
- `x0` – početna tačka pretraživanja,
- `delta_x` – veličina koraka diskretizacije prostora,
- `N` – maksimalan broj iteracija,
- `eps` – minimalna promjena vrijednosti funkcije između dvije sukcesivne iteracije,
- `T0` – početna temperatura.

Program 2.1: Definicija testne funkcije

```
function SA(f, opseg, x0, delta_x, N, eps, T0; alpha=0.995)
    x      = float.(x0)
    v      = f(x[1], x[2])
    T      = float(T0)
    x_best = copy(x)
    f_best = v
    v_prev = v

    for _ in 1:N
        # Korak 3: generisi susjedno rjesenje
        dx    = delta_x .* (2 .* rand(2) .- 1)
        x_new = clamp.(x .+ dx,
                       [opseg[1][1], opseg[2][1]],
                       [opseg[1][2], opseg[2][2]])
        v_new = f(x_new[1], x_new[2])
```

```
        dv    = v_new - v

        # Metropolisov kriterij
        if dv < 0 || rand() < exp(-abs(dv) / T)
            x, v = copy(x_new), v_new
            if v < f_best
                x_best, f_best = copy(x), v
            end
        end

        # Korak 5: hladjenje
        T *= alpha

        # Korak 4: kriterij zaustavljanja
        if abs(v - v_prev) < eps && T < 1e-6
            break
        end
        v_prev = v
    end

    return x_best, f_best
end
```

2.2 Struktura koda

Funkcija `SA` prima parametre definirane specifikacijom zadatka [3], dok `alpha` ima podrazumijevano vrijednost 0.995 . Susjedno rješenje generiše se perturbacijom trenutnog rješenja za slučajni pomak $\Delta\mathbf{x} \in (-\delta, \delta)^2$ korištenjem Julia funkcije `rand(2)`, uz `clamp.()` koji garantuje da nova tačka ostaje unutar zadanog opsega. Posebno se prati globalno najbolje pronađeno rješenje (`x_best`, `f_best`), čime se osigurava da privremeno prihvaćanje lošijeg rješenja putem Metropolisovog kriterija ne izgubi do tada pronađeni optimum. Temperatura se smanjuje geometrijskim rasporedom $T_{k+1} = \alpha \cdot T_k$ pri svakoj iteraciji, a algoritam terminira kada temperatura padne ispod tolerancije ili se dostigne maksimalni broj iteracija `N`.

2.3 Testne funkcije

Implementirana funkcija testirana je na četiri standardne optimizacijske benchmark funkcije [3], sve dvodimenzionalne oblika $f : \mathbb{R}^2 \to \mathbb{R}$, čije su Julia definicije:

Program 2.2: Definicije testnih funkcija

```
paraboloid(x1, x2)   = x1^2 + x2^2

rastrigin(x1, x2)    = 20 + x1^2 - 10*cos(2*pi*x1) +
                            x2^2 - 10*cos(2*pi*x2)

drop_wave(x1, x2)    = -(1 + cos(12*sqrt(x1^2 + x2^2))) /
                         (0.5*(x1^2 + x2^2) + 2)

holder_table(x1, x2) = -abs(sin(x1) * cos(x2) *
                          exp(abs(1 - sqrt(x1^2 + x2^2)/pi)))
```

Paraboloid

$$f_1(x_1,x_2) = x_1^2 + x_2^2 \tag{2.1}$$

Konveksna funkcija s jedinstvenim globalnim minimumom u: $\mathbf{x}^* = (0,0)$, $f_1(\mathbf{x}^*) = 0$. Koristi se kao osnovna validacija ispravnosti implementacije.

Rastriginova funkcija

$$f_2(x_1,x_2) = 20 + x_1^2 - 10\cos(2\pi x_1) + x_2^2 - 10\cos(2\pi x_2) \tag{2.2}$$

Klasična višejamna benchmark funkcija s globalnim minimumom $\mathbf{x}^* = (0,0)$, $f_2(\mathbf{x}^*) = 0$. Posjeduje veliki broj lokalnih minimuma raspoređenih na pravilnoj mreži perioda 1, što je čini najtežom funkcijom u ovom skupu testova.

Drop-Wave funkcija

$$f_3(x_1,x_2) = -\frac{1 + \cos\left(12\sqrt{x_1^2 + x_2^2}\right)}{0{,}5\,(x_1^2 + x_2^2) + 2} \tag{2.3}$$

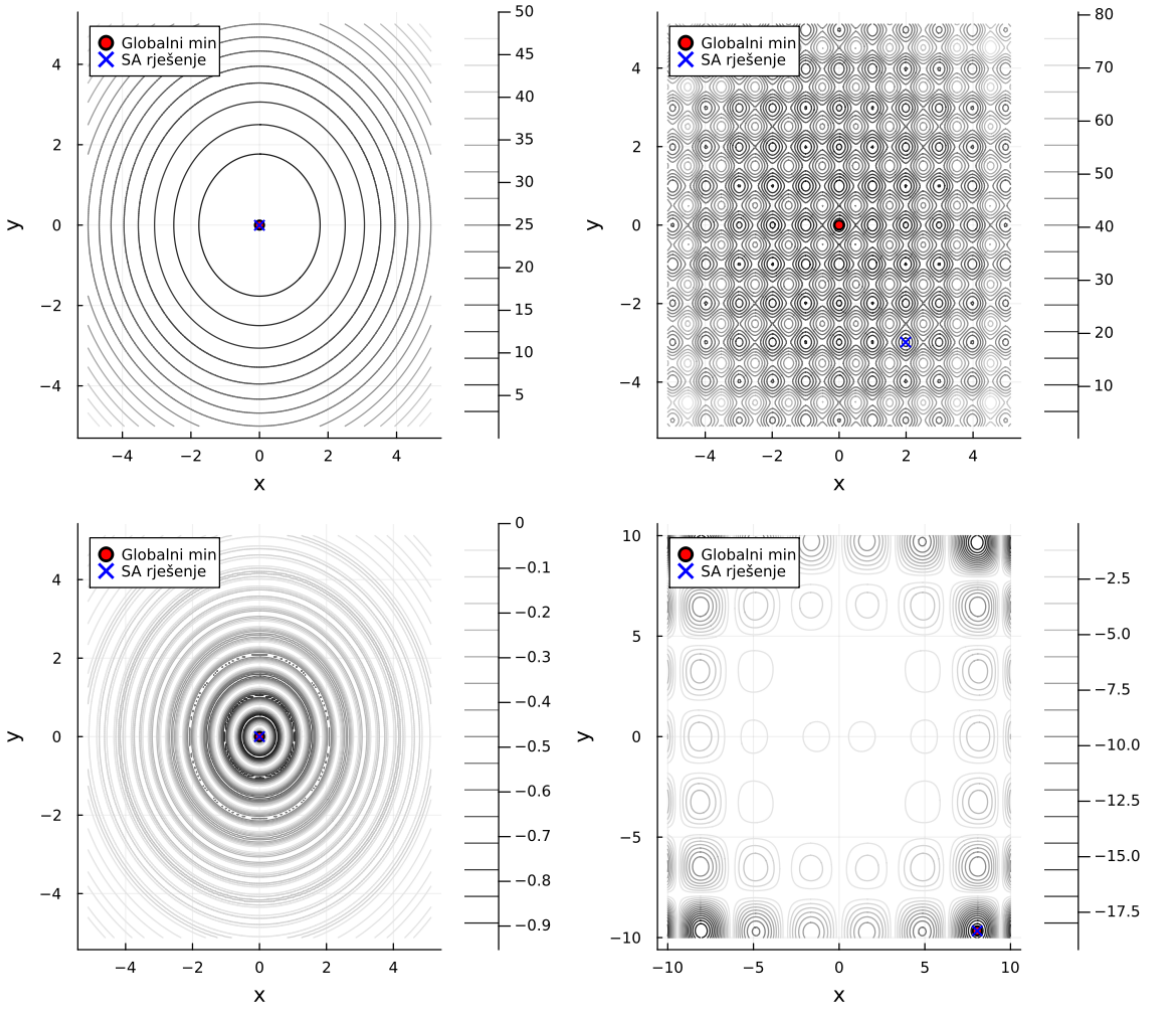
Funkcija s koncentričnim prstenovima lokalnih minimuma i globalnim minimumom u : $\mathbf{x}^* = (0,0)$, $f_1(\mathbf{x}^*) = -1$.

Holder Table funkcija

$$f_4(x_1,x_2) = -\left|\sin(x_1)\cos(x_2)\exp\left(\left|1 - \frac{\sqrt{x_1^2 + x_2^2}}{\pi}\right|\right)\right| \tag{2.4}$$

Funkcija s četiri simetrično raspoređena globalna minimuma u $(\pm 8{,}0550, \pm 9{,}6646)$, sa vrijednošću $f_4(\mathbf{x}^*) \approx -19{,}2085$.

2.4 Rezultati testiranja



Slika 2.1: Contour plotovi testnih funkcija. Crvenim kružićem označen je globalni minimum, a plavim križićem najbolje rješenje pronađeno algoritmom simuliranog kaljenja. (gore Lijevo: Paraboloid, gore Desno: Rastringova funkcija, dole lijevo: drop-wave funkcija, dole desno: Holder table funkcija)

Rezultati sva četiri testna slučaja prikazani su na slici 2.1 te u tablici 2.1. Za prve 3 funkcije posmatran je opseg (-5,5) za obje varijable, dok je za Holder Table funkciju posmatrani opseg bio (-10,10) za obje varijable. Početna tačka za oba metoda se bira randomizirano unutar opsega. Takođe u tabeli je prikazano i kakve rezultate daje jedan popularni metod za pronalazak miniumuma (Nelder-Mead), radi usporedbe sa kaljenjem.

Funkcija	T0	Alpha	SA_Diff	NM_Diff
Paraboloid	1.0	0.995	0.000068186	6.716e-10
Paraboloid	1.0	0.998	0.000036201	6.716e-10
Paraboloid	3.0	0.995	0.0000168108	6.716e-10
Paraboloid	3.0	0.998	0.0000132178	6.716e-10
Paraboloid	5.0	0.995	0.000052579	6.716e-10
Paraboloid	5.0	0.998	0.0000328166	6.716e-10
Rastrigin	1.0	0.995	31.8456	43.2352
Rastrigin	1.0	0.998	31.8388	43.2352
Rastrigin	3.0	0.995	31.8510	43.2352
Rastrigin	3.0	0.998	31.8529	43.2352
Rastrigin	5.0	0.995	31.8606	43.2352
Rastrigin	5.0	0.998	16.9150	43.2352
Drop-Wave	1.0	0.995	0.00618381	0.814157
Drop-Wave	1.0	0.998	0.00230473	0.814157
Drop-Wave	3.0	0.995	0.00553326	0.814157
Drop-Wave	3.0	0.998	0.00271886	0.814157
Drop-Wave	5.0	0.995	0.00823804	0.814157
Drop-Wave	5.0	0.998	0.00197343	0.814157
Holder Table	1.0	0.995	0.0000430246	11.4142
Holder Table	1.0	0.998	0.000131834	11.4142
Holder Table	3.0	0.995	0.00061371	11.4142
Holder Table	3.0	0.998	0.000287959	11.4142
Holder Table	5.0	0.995	0.000630948	11.4142
Holder Table	5.0	0.998	0.000170024	11.4142

Tabela 2.1: Rezultati algoritma simuliranog kaljenja (SA) i Nelder-Mead (NM) za testne funkcije. Prikazane su razlike u odnosu na globalni minimum

2.4.1 Zaključci o tačnosti

Analizom rezultata prikazanih u Tabeli 2.1 može se uočiti nekoliko važnih karakteristika u vezi tačnosti simuliranog kaljenja.

Za glatke funkcije poput Paraboloida, SA metoda je postigla veoma male razlike u odnosu na globalni minimum (5 tačih cifara). NM metoda u ovim slučajevima pronalazi bolje rješenje.

Kod Rastring funckije vidimo da SA ne daje baš njabolje rezultate, što pokazuje da i on može da zagalvi u nekom lokalnom minimumu, ali vidimo da pronalazi bolje rezulatte nego NM metoda. Nijedna od provedenih simulacija nije ni blizu traženog globalnog minimuma, pa nema smisla ni pričati o tačnosti.

Za funkcije sa oscilatornim ponašanjem, poput Drop-Wave, SA metoda je uspjela pronaći vrlo dobre aproksimacije globalnog minimuma sa malim razlikama (3 cifre tačnosti), dok NM metoda ponovo daje opet nešto slabije rješenje. Ovo pokazuje da simulirano kaljenje može biti korisno u slučajevima gdje pejzaž funkcije ima česte oscilacije i gdje determinističke metode mogu zahtijevati precizno inicijaliziranje.

Jedan od najkompjeksnihijih primjer u tabeli je funkcija Holder Table, gdje SA metoda postiže aproksimaciju koja je puno bliža globalnom minimumu u poređenju sa NM metodom, koja nije uspjela pronaći globalno rješenje zbog početnih uslova i višejamnog pejzaža funkcije. U prvoj simulaciji postignuta je tačnost od 5 cifara.

Poglavlje 3

Primjene u praksi

Simulirano kaljenje pokazuje izuzetnu fleksibilnost kao metaheuristika te se može primijeniti na širok spektar kombinatornih optimizacijskih problema koji su po prirodi NP-teški. U nastavku su opisan je jedan klasičan primjer zajedno s načinom na koji se algoritam prilagođava i josš neki primjeri upotrebe.

3.1 Problem trgovačkog putnika

Konkretan primjer primjene simuliranog kaljenja pruža *problem trgovačkog putnika* (engl. *Traveling Salesman Problem*, TSP). Trgovački putnik posjećuje N gradova poznatih koordinata (x_i, y_i), vraćajući se na kraju u grad polaska. Svaki grad smije se posjetiti samo jednom, a ruta treba biti što je moguće kraća.

Ovaj problem pripada klasi NP-kompletnih problema, čije vrijeme računanja za egzaktno rješenje raste s brojem gradova N otprilike kao $\exp(\text{const} \cdot N)$, što ga čini računski nepraktičnim već za umjereno velike N. Pored toga, TSP je karakterističan po tome što funkcija cilja E posjeduje velik broj lokalnih minimuma. U praktičnim primjenama često je dovoljno pronaći minimum koji, iako nije apsolutni, ne može biti značajno poboljšan. Upravo to postiže metoda simuliranog kaljenja, ograničavajući broj potrebnih izračuna na veličinu reda malog stepena od N [4].

3.1.1 Konfiguracija i prikaz rješenja

Gradovi se numeriraju indeksima $i = 0, 1, \ldots, N-1$, pri čemu svaki grad ima koordinate (x_i, y_i). Konfiguracija (tj. jedno rješenje) je permutacija niza $0, 1, \ldots, N-1$, interpretirana kao redoslijed kojim se gradovi posjećuju.

3.1.2 Operator susjedstva

Efikasan skup poteza za generiranje susjednih rješenja predložio je Lin [4, 5]. Potezi su dviju vrsta:

1. Dio puta se ukloni i zamijeni istim gradovima u obrnutom redoslijedu (*2-opt* operator).
2. Dio puta se ukloni i umetne između dva nasumično odabrana grada na drugom dijelu puta.

3.1.3 Funkcija cilja

U najjednostavnijem obliku problema, funkcija cilja E jednaka je ukupnoj dužini ture:

$$E = L = \sum_{i=0}^{N-1} \sqrt{(x_i - x_{i+1})^2 + (y_i - y_{i+1})^2}, \tag{3.1}$$

uz konvenciju da se tačka N poistovjećuje s tačkom 0.

Fleksibilnost metode ilustrira se sljedećim proširenjem: pretpostavimo da putnik izbjegava prelazak rijeke Mississippi. U tom slučaju svakom gradu dodjeljuje se parametar λ_i, koji iznosi $+1$ za gradove istočno od rijeke i -1 za gradove zapadno od nje. Funkcija cilja tada postaje:

$$E = \sum_{i=0}^{N-1} \left[\sqrt{(x_i - x_{i+1})^2 + (y_i - y_{i+1})^2} + \mu \, (\lambda_i - \lambda_{i+1})^2 \right], \tag{3.2}$$

gdje svaki prelazak rijeke uvodi kaznu 4μ. Algoritam pronalazi najkraći put koji izbjegava prelaske, a relativni značaj dužine puta nasuprot prelasku rijeke kontrolira se parametrom μ. Ovakav pristup može se generalizirati na problem s mnogo konfliktnih ciljeva u minimizaciji [4].

3.1.4 Raspored hlađenja

Određivanje rasporeda hlađenja zahtijeva eksperimentiranje. Preporučeni postupak je sljedeći [4]:

1. Generiše se određen broj nasumičnih permutacija kako bi se odredio tipičan raspon vrijednosti ΔE od koraka do koraka.
2. Početna temperatura T_0 postavlja se znatno iznad maksimalnog tipičnog ΔE.
3. Temperatura se smanjuje geometrijski u koracima od 10% (tj. $T_{k+1} = 0{,}9\,T_k$).
4. Svaka vrijednost temperature zadržava se dok se ne izvrši $100N$ rekonfiguracija ili $10N$ uspješnih rekonfiguracija, ovisno što nastupi prije.
5. Algoritam se zaustavlja kada daljnje smanjenje E postane dovoljno malo.

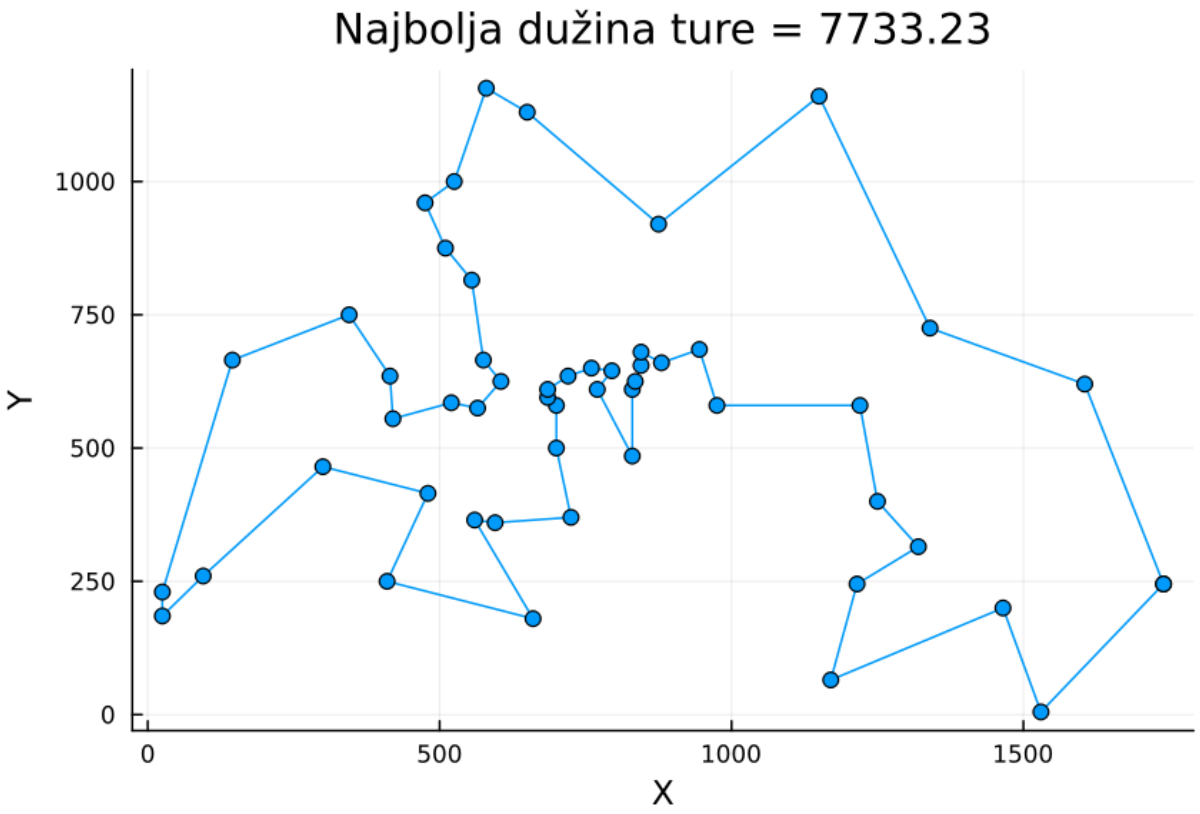
3.1.5 Testni primjer

Za instancu s $N = 1000$ nasumično generiranih gradova i primjenom 2-opt operatora, algoritam u jednoj minuti računanja na procesoru od 2,4 GHz postiže turu ukupne dužine 360 482, polazivši od inicijalne ture dužine $\approx 1{,}17 \times 10^8$. Iako dobiveno rješenje nije globalni optimum (vidljiva su određena suboptimalna ukrštanja), ono predstavlja izuzetno dobru aproksimaciju lako dobivenu direktnom primjenom simuliranog kaljenja [4].

3.1.6 Testiranje na Berlin52 instanci

Kako bi se procijenila efikasnost algoritma simuliranog kaljenja, algoritam će biti testiran na poznatoj benchmark instanci problema trgovačkog putnika Berlin52 iz biblioteke TSPLIB. Ova instanca sadrži 52 grada sa zadanim euklidskim koordinatama i često se koristi kao standard za poređenje heurističkih i metaheurističkih metoda.

Poznato optimalno rješenje za Berlin52 instancu ima ukupnu dužinu ture 7542 [6], a najbolje rješenje koje smo mi dobili u 30 pokušaja (izvršavanje trajalo nešto manje od minute) primjenom simuliranog kaljenja daje turu dužine 7733.23, što je jako dobar rezultat sa relativnom greškom od $0,025355\%$



Slika 3.1: Berlin52 tura dobijena primjenom simuliranog kaljenja

3.2 Problem ruksaka

Problem ruksaka (engl. *knapsack problem*) glasi: dat je ruksak koji može podnijeti određenu težinu i skup predmeta koji imaju težinu i vrijednost; cilj je napuniti ruksak tako da sadrži što veću vrijednost, a da pri tome ukupna težina ne premaši dozvoljeni kapacitet. Kao i TSP, problem ruksaka pripada klasi NP-kompletnih problema, što znači da egzaktno rješenje za veliki broj predmeta zahtijeva računski nepraktično dugo vrijeme i daje motivaciju za primjenu heurističkih algoritama.

Neka je na raspolaganju n predmeta, pri čemu je i-tom predmetu dodijeljena težina w_i i vrijednost v_i, a maksimalna nosivost ruksaka označena je s P. U binarnoj (0-1) varijanti problema za svaki predmet donosi se binarna odluka - uzeti ili ne uzeti, pa se rješenje predstavlja binarnim vektorom $\mathbf{x} \in \{0,1\}^n$. Optimizacijski problem formalno se zapisuje kao:

$$\max \ f(\mathbf{x}) = \sum_{i=1}^{n} x_i \cdot v_i \qquad \text{uz uvjet} \qquad \sum_{i=1}^{n} x_i \cdot w_i \leq P. \tag{3.3}$$

Za razliku od TSP-a, gdje se minimizira funkcija cilja, ovdje se traži *maksimalna* popunjenost ruksaka. Susjedna konfiguracija vektoru $\mathbf{x}$ dobiva se nasumičnom promjenom vrijednosti

na jednom nasumično odabranom mjestu k (*bit-flip*) [1]. Budući da susjedna rješenja mogu narušiti ograničenje kapaciteta, uvodi se kaznena funkcija: definira se višak težine

$$\Delta = \min\left(0,\ \sum_{i=1}^{n} x_i \cdot w_i - P\right) \tag{3.4}$$

te se evaluacija vrši nad penaliziranom funkcijom $f(\mathbf{x}) - \mu\frac{\Delta}{P}$, gdje je μ korisnički definiran parametar kazne. Na početku algoritma prihvataju se i rješenja s prekoračenjem kapaciteta (visoka vrijednost uz veliku težinu); kako temperatura opada, algoritam postepeno izbacuje teže predmete i konvergira ka dopustivom rješenju maksimalne vrijednosti. [1]

3.3 Optimizacija neuronskih mreža

Simulirano kaljenje može se primijeniti na optimizaciju neuronskih mreža na dva načina: za pronalaženje optimalnih *hiperparametara* (broj slojeva, broj neurona po sloju, stopa učenja i sl.) te za direktnu optimizaciju *težina* mreže kao alternativu gradijentnom spustu.

U prvom slučaju, svaka konfiguracija hiperparametara predstavlja jedno rješenje, a funkcija cilja je greška na skupu za validaciju. Operator perturbacije mijenja jedan hiperparametar (npr. poveća ili smanji broj neurona u jednom sloju), a SA pretražuje prostor konfiguracija izbjegavajući lokalne minimume koji su česti pri gradijentnim metodama.

U drugom slučaju, prostor pretrage su same težine mreže. SA se pokazuje posebno korisnim kada je funkcija gubitka diskontinuirana ili nediferijenatibilna, što onemogućava primjenu gradijentnih metoda. Nedostatak ovog pristupa je sporija konvergencija u poređenju s gradijentnim metodama za glatke funkcije gubitka, pa se SA u ovom kontekstu češće koristi kao dopuna nego kao zamjena gradijentnoj optimizaciji.

Neki drugi zanimljivi problemi koji se mogu rješavati pomoću SA algoritma su: bojanje grafa (engl. *graph coloring*), savijanje proteina (engl. *protein folding*), raspoređivanje sati (engl. *timetabling*), VLSI dizajn čipova, rješavanje Sudokua te optimizacija portfelja u finansijama.

Poglavlje 4

Zaključak i diskusija

Ovaj rad je analizirao heuristički metod simuliranog kaljenja s ciljem istraživanja njegovih teorijskih osnova, implementacije i primjene u optimizacijskim problemima. Simulirano kaljenje je zasnovano na principima termodinamičkog procesa kaljenja metala, a kroz fizikalnu analogiju i Metropolisov kriterij prihvatanja omogućuje efikasno pretraživanje prostora rješenja i izbjegavanje lokalnih minimuma. Pokazano je da raspored hlađenja igra ključnu ulogu u ponašanju algoritma, direktno utičući na brzinu konvergencije i kvalitet konačnog rješenja.

Implementacija algoritma u programskom jeziku Julia testirana je na nekoliko standardnih testnih funkcija. Evaluirana je sposobnost algoritma u pronalaženju globalnog minimuma te su njegove performanse upoređene s Nelder-Mead metodom. Eksperimentalni rezultati pokazuju da simulirano kaljenje nudi stabilnije rješenje u problemima s višestrukim lokalnim minimumima, iako u određenim slučajevima pokazuje nižu preciznost u odnosu na Nelder-Mead metodu.

Rad je dodatno obuhvatio nekoliko praktičnih primjena algoritma, uključujući rješavanje problema trgovačkog putnika, problema ruksaka te optimizaciju neuronskih mreža. U svakom od ovih slučajeva pokazuje se da algoritam može biti efikasan alat za optimizaciju u kompleksnim pretraživalačkim prostorima, naročito kada egzaktne metode nisu računski izvodive.

Moguća unapređenja algoritma uključuju primjenu adaptivnih tehnika hlađenja te hibridizaciju s drugim metaheurističkim metodama kako bi se poboljšala efikasnost pretrage. Bolji rezultati mogu se postići i povoljnim odabirom početnih parametara, mada ovo unapređenje uvelike ovisi o specifičnostima konkretnog problema koji se pokušava riješiti.

Literatura

[1] Delahaye, D., Chaimatanan, S., Mongeau, M., "Simulated annealing: From basics to applications", in Handbook of metaheuristics. Springer, 2018, str. 1–35.

[2] Kirkpatrick, S., Gelatt Jr, C. D., Vecchi, M. P., "Optimization by simulated annealing", science, Vol. 220, No. 4598, 1983, str. 671–680.

[3] Konjicija, S., "Predavanje 5 i laboratorijska vježba 5 – optimizacija resursa", nastavni materijal, smjer Računarstvo i informatika. 2025.

[4] Press, W. H., Numerical recipes 3rd edition: The art of scientific computing. Cambridge university press, 2007.

[5] Lin, S., "Computer solutions of the traveling salesman problem", Bell System Technical Journal, Vol. 44, No. 10, 1965, str. 2245–2269.

[6] Otman, A., Tajani, C., Abouchabaka, J., "Hybridizing psm and rsm operator for solving np-complete problems: Application to travelling salesman problem", International Journal of Computer Science Issues, Vol. 9, 03 2012.